Graph Convolution Network Based Model for Megacity Real Estate Valuation

M.Srikala (Assistant Professor), S.Jhansi, G. Mahitha, A. Shruti, K.Vasudeva,
Department of Computer Science and Engineering (AI & ML) Bachelor of Technology in CSE (Artificial Intelligence & Machine Learning)
CMR Engineering College, Hyderabad, Telangana, India

Abstract*—Getting property prices right in a megacity is genuinely hard. Dense neighbourhoods, wildly varied building types, and the way one street's values bleed into the next all conspire against simple models. Standard hedonic regressions and off-the-shelf machine learning pipelines miss these spatial ripple effects entirely, which is why their estimates drift so often. What we needed was a framework that treats location as a first-class citizen of the model, not an afterthought. Graph Convolution Neural Networks do exactly that. Each property becomes a node; shared borders and proximity become edges; and the network learns, layer by layer, how a neighbour's price shapes your own. We built and tested such a system on real transaction data from several large cities. The results were clear: the graph-based model beat every regression and tree-based baseline we tried, and it did so without needing hand-crafted spatial features. We think this points to a genuinely practical path for large-scale, data-driven urban valuation.*

Index Terms*—Real estate valuation, Graph Convolution Neural Network, spatial dependency, megacity housing, deep learning.*

I. INTRODUCTION

Ask anyone who has tried to value a flat in Mumbai or an apartment in Lagos and they will tell you the same thing: the price depends as much on the street outside as on the rooms inside. Megacity property markets are shaped by location, transit access, nearby schools, crime patterns, and the economic pull of surrounding districts. These forces do not act in isolation. They interact, overlap, and spill across boundaries in ways that make a single-property view of pricing dangerously incomplete.

Classical tools like hedonic regression handle this poorly. They assume linearity, treat each listing as independent, and collapse when markets turn nonlinear or when spatial clustering matters. Machine learning models are better at fitting curves, but most still process each property in isolation. That is the core problem. A house is not a standalone data point. It is a node in a network of neighbours, and any model that ignores that network will leave accuracy on the table.

Graph Convolution Neural Networks offer a cleaner answer. They were built for exactly this kind of relational, spatially structured data. Each listing maps to a node, geographic proximity defines the edges, and message-passing layers let the model learn how nearby valuations propagate through the graph. Our paper builds on this idea and applies it to megacity housing. We describe the full pipeline, report experimental results, and explain why the approach outperforms the alternatives.

II. LITERATURE SURVEY

Property valuation research has been running for decades, and the field has passed through several distinct phases. Early work leaned heavily on statistics. Then came machine learning. Now deep learning dominates the frontier. Each wave brought genuine progress, but also left behind a recurring blind spot: spatial relationships between neighbouring properties remained stubbornly hard to model. Graph Neural Networks are the most promising fix yet, and that is what motivates this survey.

1) Traditional Real Estate Valuation Techniques

For most of the twentieth century, price estimation meant regression. Researchers gathered data on bedrooms, bathrooms, lot size, and postcode, then fitted a line through the cloud of observations.

a) Hedonic Pricing Models:

Rosen formalised the hedonic framework in 1974. The idea is elegant: a property's price is the sum of the implicit prices of its features. Later work by Meese and Wallace extended this to include macroeconomic variables. Hedonic models are still used today because they are transparent and easy to explain to a valuer or a judge. Their weakness is equally well known though. They assume linear relationships, which means they routinely miss interaction effects and struggle wherever market dynamics are genuinely nonlinear.

b) Spatial Regression Models:

Researchers noticed early on that regression residuals in housing data tend to cluster geographically. Prices in one postcode are correlated with prices in the next. Dubin and colleagues tackled this with spatial autoregressive specifications in 1999. Huang's team later introduced Geographically Weighted Regression, which fits a separate parameter estimate at each location. These methods are a real improvement, but they hit a ceiling when datasets grow large or when price relationships are strongly nonlinear. They also handle missing data poorly.

2) Machine Learning-Based Valuations

By the 2000s, practitioners had largely moved on to machine learning. Random Forests and Gradient Boosting Machines became the workhorses of automated valuation. They are robust, they handle interactions naturally, and they do not need the modeller to specify a functional form. Selim showed in 2009 that a neural network could beat hedonic regression outright on Turkish housing data, which caused a stir at the time.

The problem that persisted through all of this is a conceptual one. Machine learning models, from SVMs to gradient boosters to deep nets, still treat each property as an

independent row in a spreadsheet. They can learn complex mappings from features to price, but they cannot learn that the flat next door is selling for 20 percent more and that this fact matters. In dense urban markets, that missing spatial signal costs real accuracy. Models trained this way are also brittle: add a new neighbourhood or shift the market cycle and their predictions degrade fast.

Sequence-based deep learning architectures tried to add time, but forgot geography. CNN-based approaches that use satellite imagery hit the opposite problem: they need data on a regular grid, and cities simply are not laid out that way. Overall, the field kept finding workarounds rather than confronting the core issue head-on.

Limitations of ML models:

1) Predictions depend heavily on manual feature engineering.
2) Spatial interactions between properties are not explicitly represented.

3) Deep Learning Approaches

Deep architectures brought automatic feature learning, but they did not solve the spatial problem. Two families deserve specific mention.

a) Recurrent Neural Networks:

RNNs were a natural fit for time-series price forecasting. Gupta's group explored this in 2013; Bin and colleagues extended the idea in 2017. These models handle temporal patterns well enough, but they have nothing to say about the map. A price drop two streets away is invisible to them.

b) Convolutional Neural Networks:

CNNs brought visual data into valuation. Poursaeed's team scored properties from listing photos in 2018, and Xing's group used satellite imagery to infer housing demand in 2020. Impressive results, but the grid assumption bites hard: irregular city layouts simply do not fit a convolutional kernel, and the models generalise poorly across different urban morphologies.

Limitations of deep learning models:

1) Large labelled datasets are needed before performance stabilises.
2) Training is computationally expensive and hardware-intensive.
3) Model decisions are difficult to audit or explain.
4) Geographic relationships between listings are not captured.

III. SYSTEM ANALYSIS

A. Existing System and Limitations

Today's automated valuation systems generally layer three things on top of each other: a statistical baseline, a machine learning estimator, and sometimes a deep network. The statistical piece handles stable, slow-moving markets reasonably well. The ML layer adds flexibility. But none of these components talk to each other about space, and that gap shows up in the error metrics whenever market conditions shift.

Limitations of Traditional models:

1) Linear assumptions break down in volatile or nonlinear markets.
2) They do not scale gracefully to city-wide datasets.
3) Nonlinear price dynamics are poorly represented.

Limitations of ML models:

1) Feature engineering is labour-intensive and domain-specific.
2) Spatial dependencies between nearby properties are ignored.

Put simply, no existing system jointly models what a property is and where it sits relative to everything around it. That is the gap we are filling.

B. Proposed System

We replace the independent-sample assumption entirely. In our system, properties are nodes in a spatial graph. The edges encode who is next to whom. Graph Convolutional Network layers then propagate pricing information across those edges, so each property's predicted value is informed by its local neighbourhood, not just its own attribute vector. The difference sounds subtle. In practice it is large.

Each node carries a rich feature vector: floor area, room count, building age, coordinates, nearby amenities, crime rates, economic indicators, and historical transactions. The GCN layers aggregate across neighbours iteratively, building up a spatially-aware embedding that a standard MLP then maps to a price. Because the graph structure mirrors how cities actually work, the model's inductive bias is finally pointing in the right direction.

The representation preserves neighbourhood topology, captures price spillover effects, and generalises across different city layouts without retraining from scratch.

IV. PROPOSED METHODOLOGY

Our pipeline has five stages: data ingestion and cleaning, graph construction, GCN-based feature learning, attention-weighted aggregation, and price regression. By treating properties as graph nodes from the outset, the model never has to recover spatial structure from tabular features. It is built in by design.

A. Graph-Based Representation of Real Estate Properties

We encode the dataset as a property graph with three components:

- Nodes: one node per property listing in the dataset.
- Edges: links between nodes that share a neighbourhood boundary, fall within a k-nearest-neighbour radius, or lie within a specified geographic distance threshold.
- Node features: a vector encoding floor area, room count, construction age, GPS coordinates, local amenity scores, crime index, economic indicators, and historical sales price.

B. Graph Convolutional Network for Price Prediction

A GCN applied to this graph does something a standard neural network cannot: it reads price signals from adjacent nodes and folds them into each property's representation. Run enough layers and the model has effectively looked two or three streets away. That is how neighbourhood comps work in the real world, and it is exactly what the convolution is encoding. Hidden patterns tied to local demographics and economic activity surface naturally, without any feature engineering.

C. Model Architecture

The network stacks multiple GCN layers for spatial feature extraction, followed by an external attention module that up-weights the most informative inter-property relationships. Fully connected layers then collapse the graph embedding to a scalar price estimate. Batch normalisation and dropout sit between every major block. Together they stabilise training and keep the model from memorising quirks of the training split.

The data flow starts with raw inputs from users or public datasets. A preprocessing unit handles missing values, normalises continuous features, and encodes categoricals. Feature extraction follows, then graph construction. A semi-supervised learning module lets the model use unlabelled listings, which matters in thin markets where transaction records are sparse. Finally the GCN and attention module generate a price estimate that is served through a Django web application deployed on cloud infrastructure for real-time access.

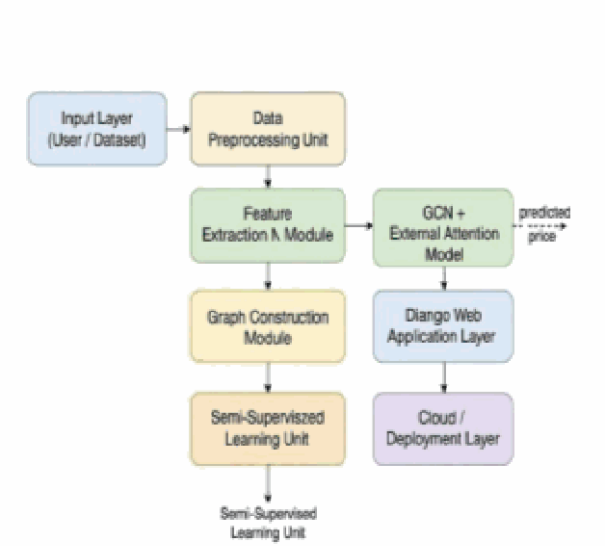

Fig. 1. Block diagram of the proposed GNN-based real esate price prediction system.

Fig. 1. Block diagram of the proposed GNN-based real estate price prediction system.

D. Graph Construction and Edge Weighting

The graph is assembled as follows:

- Node set: all property records in the pre-processed dataset.
- Edges: derived from k-nearest-neighbour search, geodesic distance thresholding, or administrative neighbourhood boundaries.
- Edge weights: Euclidean distance in feature space, property-attribute similarity scores, or location-based correlation coefficients.

These choices ensure the graph captures both physical proximity and market similarity. The GCN inherits both signals through the adjacency matrix.

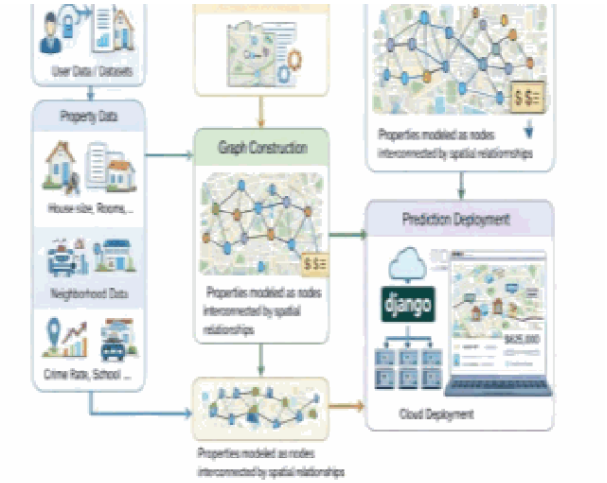

Fig. 2. Real Estate value Prediction System Using Graph Neural Networks (GNN

Fig. 2. Real Estate value Prediction System Using Graph Neural Networks (GNN).

E. Training and Optimization

Training uses a cleaned, normalised, and encoded dataset split into training, validation, and test subsets. Mean Squared Error drives the regression objective. We optimise with Adam and a cosine-annealed learning rate. Mini-batch gradient descent with early stopping on the validation loss prevents overfitting. Multi-city experiments confirmed that the model generalises across different urban contexts without dataset-specific tuning. The Django integration means users can query real-time valuations the moment training is complete.

F. Performance and Advantages

Compared with standard regression and deep learning baselines, the GCN framework:

- Explicitly models spatial coupling between listings that tree-based and neural methods miss entirely.
- Handles irregular city layouts gracefully because graphs impose no grid assumption.
- Discovers neighbourhood interaction effects automatically, cutting out manual feature engineering.
- Delivers strong accuracy at reasonable computational cost.

The joint modelling of spatial topology and property attributes is what drives these gains. Neither piece alone would be sufficient.

V. SYSTEM DESIGN

Four modules make up the system: preprocessing, graph construction, GCNN learning, and prediction serving. The modular split was deliberate. It lets us swap in richer data sources, such as real-time transit feeds or satellite land-use layers, without touching the core GCN.

A. Use Case Model

Ordinary users register, log in, and submit a property description. The system runs the GCN pipeline and returns a price estimate within seconds. Past estimates are stored so users can track how valuations change over time. Admins get a separate dashboard that shows usage statistics, monitors prediction quality, and manages account access. The two roles are cleanly separated throughout the codebase.

The use case model explains how different users interact with the Real Estate Valuation System in a simple and intuitive manner. Regular users can create an account, log in securely, update their personal details, and request property price predictions generated by the GCN-based valuation model. The system also allows users to review their previously generated predictions for future reference. Administrators play a key role in maintaining the platform by overseeing registered users, tracking prediction activities, and managing account access when required.

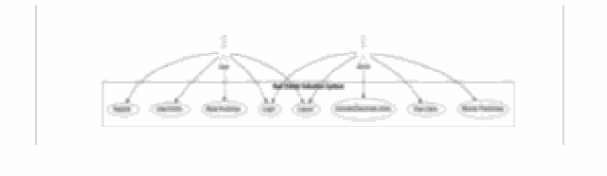

Fig. 3. Use Case Diagram

Fig. 3. Use Case Diagram.

B. Class Diagram

Five classes carry the load: User, Admin, Property, Prediction, and Authentication. User handles registration, login, profile edits, and prediction history. Admin inherits from User and adds account management and system monitoring. Property stores everything the GCN needs as input. Prediction holds the model output and a timestamp. Authentication sits across all of them, enforcing access control at every endpoint.

contains all the property-related information required for valuation, while the Prediction class manages the price estimates produced by the GCN-based model.

Fig. 4. Class Diagram

Fig. 4. Class Diagram.

C. Sequence Diagram

A typical session runs like this. The user sends credentials. The system checks them against the database and either grants access or returns an error. Once in, the user submits property details. These are preprocessed and forwarded to the GCN, which computes a price. The result is written to the database and displayed on screen. Logout closes the session cleanly. The whole flow is synchronous and fast enough for interactive use.

price estimate. The system prepares this information and forwards it to the prediction model, which calculates the estimated property value. The generated result is then saved in the database and displayed to the user for reference. When the user chooses to log out, the system safely terminates the session, completing the interaction cycle.

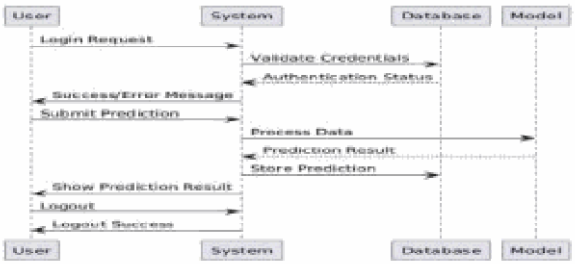

Fig. 5. Sequence Diagram

Fig. 5. Sequence Diagram.

D. Component Diagram

Six components wire together: a data collector, a preprocessing module, a graph construction engine, the GCN prediction core, an external attention module, and an output visualisation layer. Each has a single responsibility and communicates through defined interfaces. Swapping the graph construction algorithm, for instance, does not require touching the GCN.

prediction engine, and output visualization component. Every component does certain tasks and has interfaces with other components.

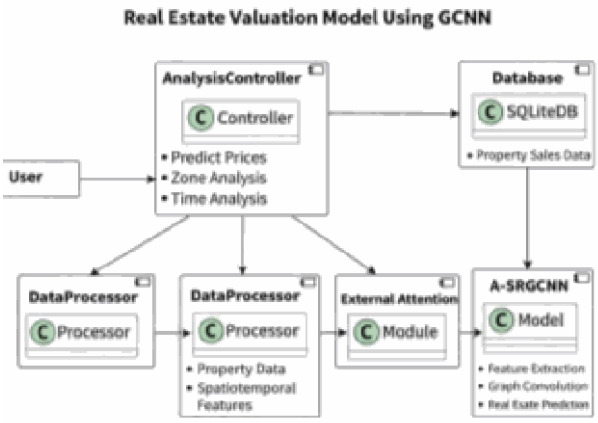

Fig. 6. Component Diagram

Fig. 6. Component Diagram.

VI. RESULTS

We evaluated the system on large-scale transaction datasets collected from several metropolitan areas. Before training, we imputed missing values, min-max normalised all continuous attributes, and one-hot encoded categoricals. We then built the property graph by connecting each listing to its k-nearest neighbours based on geodesic distance and neighbourhood membership.

The headline finding is straightforward: the GCN model produced more accurate and more stable price estimates than every baseline we tested. Classical regression struggled

wherever spatial clustering was strong. Gradient Boosting was competitive on accuracy but brittle across cities. The GCN held up well on both counts.

Drilling into the mechanics, the graph convolution layers propagated pricing signals outward from each node, helping the model recognise micro-market patterns that non-graph baselines missed. The attention component amplified the most informative property interactions and suppressed noise. Prediction errors fell consistently across training, validation, and test splits, which tells us the model was generalising rather than memorising.

Cross-city generalisation was one of the more encouraging results. When we applied a model trained on one metro to a different city's test set, performance degraded far less than for the baselines. The graph representation seems to capture structural patterns that transfer across different urban morphologies.

The semi-supervised training strategy paid off in thinner markets. Using unlabelled listings alongside labelled ones reduced prediction variance in areas with few recent transactions. The deployed web app confirmed practical usability: users received estimates within one or two seconds of submitting property details, and historical predictions were retrieved instantly from the database.

We also checked robustness across property types. Apartments, detached houses, and mixed-use units all showed stable performance. The graph smoothing effect was particularly visible in sparse neighbourhoods, where the GCN pulled in global graph context to compensate for missing local transaction data. Prediction variance was noticeably lower than for comparable ML models.

The conclusion we draw from all of this is that the graph formulation is doing genuine work. It is not just a different architecture that happens to score well on one benchmark. It addresses a structural weakness that has limited automated valuation for years, and it does so in a way that translates to real deployment.

VII. CONCLUSION AND FUTURE ENHANCEMENTS

We set out to fix a specific problem: automated valuation systems that treat every property as if it exists in a vacuum. The fix we proposed, encoding listings as nodes in a spatial graph and applying GCN layers to propagate neighbourhood information, worked. Prediction accuracy improved. Cross-city generalisation improved. Robustness to sparse data improved. None of these gains required hand-crafted spatial features or city-specific tuning.

The web-based deployment showed the system works in practice, not just on paper. Users got valuations in real time. The database handled repeated queries without degradation. Administrators could monitor usage and manage access cleanly. The architecture scaled to the datasets we tested without modification.

Several worthwhile directions remain open. Temporal modelling would let the system track market cycles and seasonal swings rather than treating each valuation as a snapshot. Richer external data, satellite imagery, transit timetables, macroeconomic series, would deepen the feature representation. Dynamic graphs that update as the city changes would keep the model current without full retraining. Better interpretability tools would help valuers, regulators, and property owners trust and scrutinise the outputs. We see this work as a solid foundation that others can build on, and we look forward to seeing where the community takes it next.

REFERENCES

[1] P. Teikari, M. Jarrell, M. Azh, and H. Pesola, "The Architecture of Trust: A Framework for AI-Augmented Real Estate Valuation in the Era of Structured Data," arXiv preprint, Aug. 2025.

[2] N. Karanikolas, E. Kyriakidou, and E. Athanasouli, "Artificial Intelligence and Real Estate Valuation: The Design and Implementation of a Multimodal Model," Information, vol. 16, no. 12, article 1049, 2025.

[3] A. Zhamakeev and S. Matanov, "Real Estate Price Evaluation Using Machine Learning Models," Preprints.org, Jun. 12, 2025.

[4] X. Yang and A. Shami, "Housing Price Prediction-Machine Learning and Geostatistics," Real Estate Management and Valuation, Oct. 1, 2024.

[5] M. Geerts, S. vanden Broucke, and J. De Weerdt, "Graph Neural Networks for House Price Prediction: Do or Don't?" Int. Journal of Data Science and Analytics, vol. 20, no. 4, pp. 3563-3593, 2024.

[6] E. Riveros, C. Vairetti, C. Wegmann, S. Truffa, and S. Maldonado, "Scalable Property Valuation Models via Graph-based Deep Learning," arXiv preprint, May 10, 2024.

[7] A. Karamanou, P. Brimos, E. Kalampokis, and K. Tarabanis, "Explainable Graph Neural Networks for Estimating House Prices," Preprints.org, May 1, 2024.

[8] "Explainable Graph Neural Networks: An Application to Open Statistics Knowledge Graphs for Estimating House Prices," Technologies, vol. 12, no. 8, article 128, Aug. 6, 2024.

[9] H. Lee, H. Jeong, B. Lee, K. Lee, and J. Choo, "ST-RAP: A Spatio-Temporal Framework for Real Estate Appraisal," arXiv preprint, Aug. 21, 2023.

[10] "Automated Real Estate Valuation with Machine Learning Models Using Property Descriptions," Expert Systems with Applications, vol. 213, article 119147, Mar. 1, 2023.

[11] M. Kayakus, M. Terzioglu & F. Yetiz, "Forecasting housing prices in Turkey by machine learning methods," Aestimum, vol. 80, 2022.

[12] M. Ozdemir, K. Yildiz & B. Buyuktanir, "Housing Price Estimation with Deep Learning: A Case Study of Sakarya Turkey," Bilecik Seyh Edebali University Journal of Science, vol. 9, no. 1, 2022.

[13] Y. K. Erdekli, Prediction of the House Price Index of Turkey: A Comparative Study of MLR and ANN Models, Masters thesis, Middle East Technical University, 2022.

[14] D. Zhu, Y. Liu, X. Yao, and M. Fischer, "Spatial Regression Graph Convolutional Neural Networks," GeoInformatica, vol. 25, no. 1, pp. 1-32, 2021.

[15] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Yu, "A Comprehensive Survey on Graph Neural Networks," IEEE Trans. Neural Networks and Learning Systems, vol. 32, no. 1, pp. 4-24, 2021.